

GKK Interns - Complete Admin & Operations Guide

1. Admin System Sections & Features

- **Active Interns:** Manage currently enrolled interns and their profiles.
- **Activity Logs:** Monitor actions taken within the system for audit purposes.
- **Announcements:** Broadcast messages and updates to interns.
- **Applications:** Review and accept/reject new internship applications.
- **Availability:** Set and manage administrator schedules and meeting slots.
- **Batches:** Organize interns into batches for streamlined coordination.
- **Custom Email:** Send tailored emails directly to individual or groups of interns.
- **Invitations:** Manage invite codes or direct invitations for new users.
- **Meetings:** Schedule and oversee video sessions and check-ins.
- **Payments:** Track intern payments, subscriptions, and verify unlocked features.
- **Projects:** Assign, track, and review project milestones.
- **Push Notifications:** Dispatch real-time mobile push notifications.
- **QR Codes:** Generate QR codes for quick mobile app access or attendance.
- **Resources:** Upload and manage learning materials and guides.
- **Rewards:** Issue certificates and manage the internal points system.
- **Submissions:** Evaluate intern project submissions and assignment uploads.
- **Teams:** Form and manage team collaborations and group projects.

2. Critical Infrastructure & Credentials

SMTP Email Configuration

The primary automated email uses Nodemailer with a dedicated Gmail account:

Email User: noreplay.gkk26@gmail.com
App Password: yutfmttgqvlqsm

Payment Gateway (Razorpay Live)

The platform is integrated with Razorpay for handling Rs. 499 application fees:

Key ID: rzp_live_SJUXJsWdw9cbj6
Key Secret: daHrwJXUB7kWNq96cyu0YI63

Supabase Edge Functions & Auth

Project configuration for database and authentication integration:

VITE_SUPABASE_URL: https://hjpsyxqakzrhvzegehtm.supabase.co
VITE_SUPABASE_ANON_KEY: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Imhqcn5eHFa3pyaHZ6ZwlaHRtIiwicm9sZSI6ImFub24iLCJpYXQiOiE3Njg3NTI5NjMsImV4cCI6MjA4NDMyODk2M30.eyJ0b2p0b3RkbnZHYtQ2q0Yl1vZ5y0QZPRrkHC0wAU

Firebase Admin SDK (Mobile Push Notifications)

Stored in AdminMobileApp/service-account.json:

Project ID: gkk-admin-app
Client Email: firebase-adminsdk-fbsvc@gkk-admin-app.iam.gserviceaccount.com
Client ID: 104526367320267631517
Private Key ID: d963d11509316b6c9b7f31dfa1cf4a812eafdbdc

3. Updating & Pushing Mobile Apps

Mobile releases are automated via Node.js scripts handling the GitHub Releases API.

Deployment Process:

1. Update the version number in app_config.json under the respective app directory (AdminMobileApp / InternMobileApp).
2. Build the Android APK using Flutter:
flutter build apk --release
3. Wait for the build to create the APK at build/app/outputs/flutter-apk/app-release.apk.
4. Run the release automation script:
node release_github.js
5. The APK will be uploaded to GitHub releases automatically.

release_github.js (Admin Release Script):

```
import fs from 'fs'; import path from 'path'; import { Octokit } from '@octokit/rest';  
// 1. GitHub Repository Info (admin-specific private repo) const OWNER = 'noreplaygkk26-png';  
const REPO = 'gkk-admin-releases'; // 2. GitHub Personal Access Token (PAT)  
const GITHUB_TOKEN = 'ghp_zfbgJpQEApCpDRHw6Bl16d02DVdl5ER4RNDUX'; // 3. File Paths  
const APP_CONFIG_PATH = './app_config.json';  
const APK_PATH = './build/app/outputs/flutter-apk/app-release.apk'; async function createRelease() {  
  try {  
    const appConfig = JSON.parse(fs.readFileSync(APP_CONFIG_PATH, 'utf8'));  
    const tagName = `admin-v${appConfig.latest_version}`; const releaseName = `Admin Re  
lease ${appConfig.latest_version} (Build ${appConfig.required_build_number})`;  
    const octokit = new Octokit({ auth: GITHUB_TOKEN });  
    const { data: release } = await octokit.rest.repos.createRelease({  
      owner: OWNER, repo: REPO, tag_name: tagName, name: releaseName,  
      body: `Admin app release v${appConfig.latest_version}.`,  
      draft: false, prerelease: false,  
    });
```

```

const fileName = path.basename(APK_PATH);
const fileContent = fs.readFileSync(APK_PATH);
await octokit.rest.repos.uploadReleaseAsset({
  owner: OWNER, repo: REPO, release_id: release.id,
  name: fileName, data: fileContent, headers: { 'content-type': 'application/
vnd.android.package-archive', 'content-length': fileContent.length },
});
console.log(`Release created: ${release.html_url}`); } catch (error) {
console.error('Release failed:', error.message); } } createRelease();

```

4. Production Deployment for Websites

Web deployment consolidates four React apps into a single structure for Netlify.

Web Deployment Steps:

1. Open the terminal in the root project directory (root folder).
2. Run the deployment batch file: `deploy.bat`
3. Wait for it to build Bento Form, Dashboard, GKK-HIRE-MAIN, and Community Chat into `production_dist`.
4. Deploy the folder to Netlify using the CLI: `netlify deploy --prod`
5. Select `production_dist` as the publish directory when prompted.

deploy.bat (Web Deployment Script):

```

@echo off setlocal enabledelayedexpansion set ROOT=%~dp0 pushd "%ROOT%"
if exist "production_dist" ( rmdir /s /q "production_dist" ) mkdir "production_dist"
pushd "gkk-bento-form" & call npm run build & popd pushd "Dashboard" & call npm run build & popd
pushd "GKK-HIRE-MAIN" & call npm run build & popd pushd "community-chat" & call npm run build & popd
robocopy "GKK-HIRE-MAIN\dist" "production_dist" /E /NFL /NDL /NJH /NJS /NC /NS >nul
mkdir "production_dist\dashboard"
robocopy "Dashboard\dist" "production_dist\dashboard" /E /NFL /NDL /NJH /NJS /NC /NS >nul robocopy "
production_dist\dashboard\admin" "production_dist\admin" /E /MOVE /NFL /NDL /NJH /NJS /NC /NS >nul
robocopy "production_dist\dashboard\css" "production_dist\css" /E /MOVE /NFL /NDL /NJH /NJS /NC /NS
>nul robocopy "production_dist\dashboard\js" "production_dist\js" /E /MOVE /NFL /NDL /NJH /NJS /NC
/NS >nul robocopy "production_dist\dashboard\user" "production_dist\user" /E /MOVE /NFL /NDL /NJH
/NJS /NC /NS >nul robocopy "production_dist\dashboard\assets" "production_dist\assets" "gkk-intern-
logo.png" "bubblesort-logo.png" /NFL /NDL /NJH /NJS /NC /NS >nul
mkdir "production_dist\dashboard\apply" robocopy "gkk-bento-form\dist" "production_dist\dashboard\ap
ply" /E /NFL /NDL /NJH /NJS /NC /NS >nul mkdir "production_dist\community-chat"
robocopy "community-chat\dist" "production_dist\community-chat" /E /NFL /NDL /NJH /NJS /NC /NS >nul
( echo /supabase-main/* https://hjpsyxqakzrhvzegehtm.supabase.co/:splat 200!
echo /supabase-chat/* https://mwnpwuxrbaousgwgoico.supabase.co/:splat 200!
echo /dashboard/admin/* /admin/:splat 301 echo /dashboard/css/* /css/:splat 301
echo /dashboard/js/* /js/:splat 301
echo /dashboard/apply/* /dashboard/apply/index.html 200
echo /Dashboard/* /dashboard/:splat 301
echo /dashboard/* /dashboard/index.html 200
echo /community-chat/* /community-chat/index.html 200
echo /* /index.html 200 ) > "production_dist\_redirects" popd

```

5. Deploying Supabase Edge Functions

Edge functions are handled by a separate deployment batch script.

deploy_functions.bat (Snippet):

```
@echo off setlocal echo Deploying Supabase Edge Functions... pushd "%~dp0"
call supabase functions deploy create-profile --no-verify-jwt
call supabase functions deploy send-otp --no-verify-jwt
call supabase functions deploy send-decision-email --no-verify-jwt
call supabase functions deploy send-application-email --no-verify-jwt
call supabase functions deploy initiate-payment --no-verify-jwt
call supabase functions deploy razorpay-order --no-verify-jwt
call supabase functions deploy razorpay-webhook --no-verify-jwt
echo CRITICAL: YOU MUST CONFIGURE THE HOOK IN SUPABASE DASHBOARD
echo 1. Go to Authentication -> Hooks echo 2. Enable "Send Email Hook" (or "Custom Email Provider")
echo 3. Select "Function" -> "send-otp"
echo    (Or paste the URL: https://.supabase.co/functions/v1/send-otp) echo 4. Save
echo Also ensure Razorpay webhook is configured to:
echo https://.supabase.co/functions/v1/razorpay-webhook popd endlocal
```